

云聚园区 智享未来

“华为云杯” 2023人工智能应用创新大赛

基于 Benders 分解的环形生产线平衡：
自顶向下和自底向上方法

郭鹏辉 (Min), <https://guo.ph>
苏州工业园区人工智能产业园, 2023-9-26



目录

01

团队介绍

02

问题分析

03

算法设计

04

数值结果

05

结论与展望



01 / 团队介绍

“华为云杯”2023人工智能应用创新大赛

团队介绍



南京航空航天大学
NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS

经济与管理学院

郭鹏辉 博士生

Min 团队（初赛第二）

南京航空航天大学经济与管理学院，智能决策与风险分析研究所

- 专业：管理科学与工程（工业工程）
- 研究兴趣：整数规划 和随机规划及其 精确算法 在物流和 生产系统 中的应用
- 个人主页：<https://guo.ph>

02 / 问题分析

“华为云杯”2023人工智能应用创新大赛

问题背景

生产线及生产线平衡



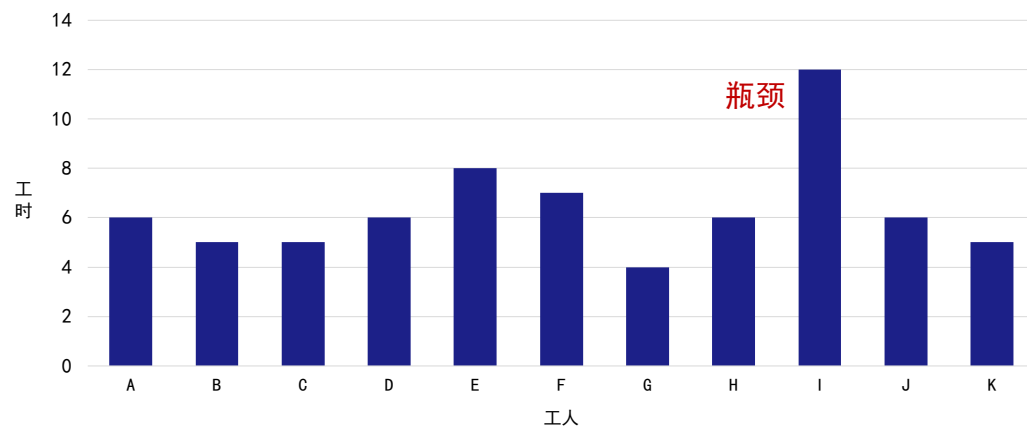
福特汽车生产线（1913年）



亨利·福特和T型车



人机协作的现代化生产线

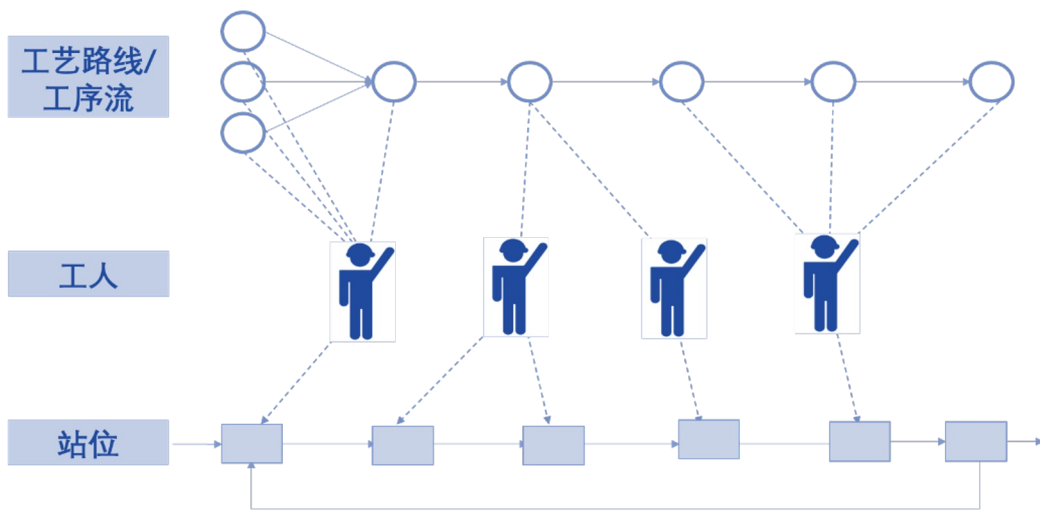


“生产线平衡”是对生产的全部工序进行平均化、均衡化，调整各工序或工位的作业负荷或工作量，使各工序的作业时间尽可能相近或相等，最终消除各种等待浪费现象，达到生产效率最大化的技术手段与方法。

消除瓶颈工序，提高人均产量，减少在制品数量，提高工作效率，实现准时制生产。

问题分析

赛题回顾



赛题简述

- 确定工序、工人、工位、和设备的分配关系。
- 满足工艺规则约束、人员技能约束、工位机器约束、节拍均衡约束、重复入站、和最大转圈数量约束。
- 生成执行工序的工位、人员、和顺序，最小化评价指标（节拍均衡）。

关键点及难点

①「数学模型规模巨大」

混合整数规划（Mixed-Integer Programming, MIP）模型的规模与 $|\text{工序}| \times |\text{工人}| \times |\text{站位}| \times |\text{圈数}|$ 成比例（几十万）。且调度 / 排序问题的线性松弛较弱，使得难以找到可行解。

②「环形生产线建模」

环形生产线平衡（Circular Assembly Line Balancing）是本次比赛首次提出的新的生产线平衡问题（Assembly Line Balancing Problem）^[1]，无成熟建模方式。

③「工序拆分建模」

工序拆分是赛题的关键约束，部分算例在不允许工序拆分时无可行解。如何对工序拆分建模，并保证所设计的算法具有通用性是赛题的又一关键点。

④「重复入站建模」

重复入站是赛题的关键约束，与“环形生产线”有密切联系，无成熟建模方式。

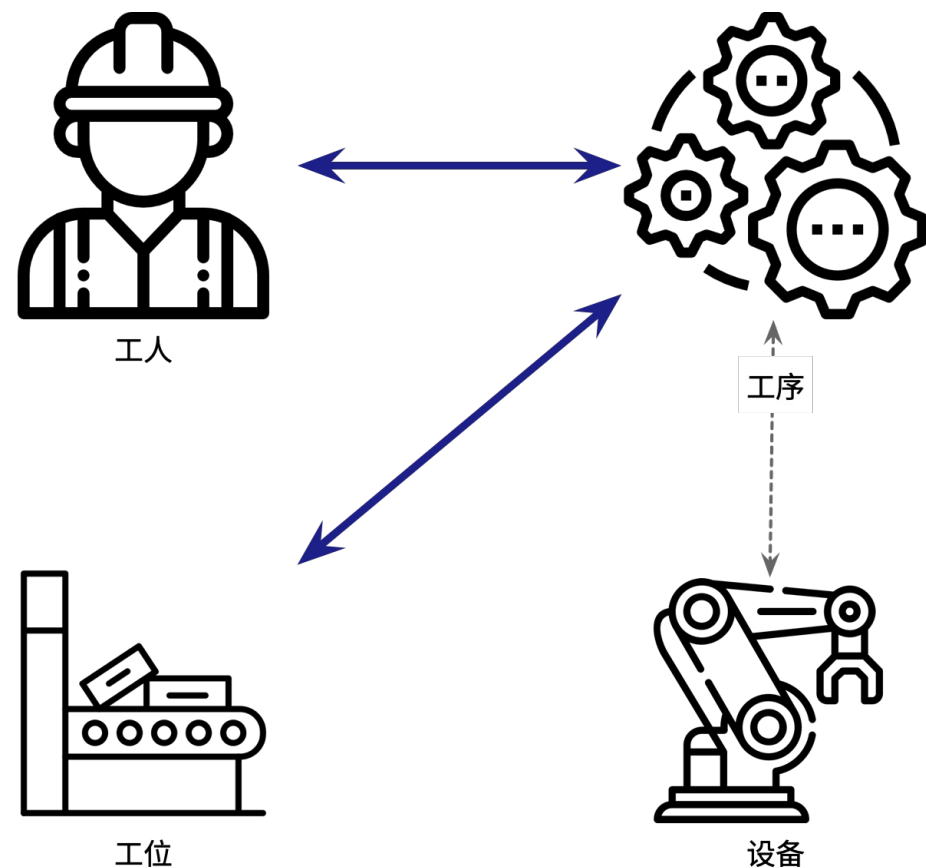
问题分析

赛题回顾

- 题目中的分配关系包括：
 - (1) 工序 $\leftrightarrow$ 工人, (2) 工序 $\leftrightarrow$ 工位,
 - (3) 工人 $\leftrightarrow$ 工位, (4) 设备 $\leftrightarrow$ 工位。
- 可以发现：
 1. 前三者任意两者确定, 第三者也相应确定
 2. 「设备 $\leftrightarrow$ 工位」隐含于「工序 $\leftrightarrow$ 工位」
- 因此可只考虑分配关系：
 - 工序 $\leftrightarrow$ 工人 (目标函数)
 - 工序 $\leftrightarrow$ 工位 (可行性)

💡 当「工序 $\leftrightarrow$ 工人」和「工序 $\leftrightarrow$ 工位」中一组确定下来, 求解另一组变得相对容易。

⇒ Benders 分解 (Benders decomposition) [2,3,4]



03 / 算法设计

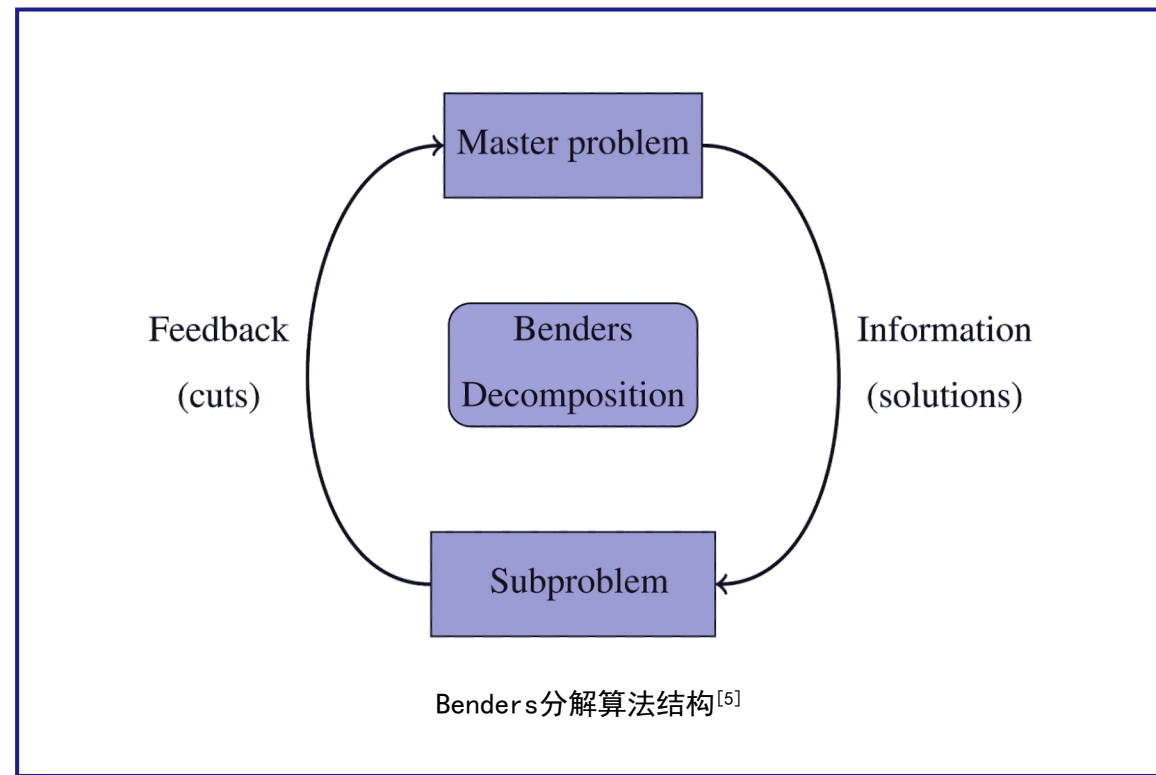
“华为云杯”2023人工智能应用创新大赛

算法设计

Benders 分解概述

Benders 分解的基本思想是将一个大型的优化问题分解为两个更小、更易于处理的子问题。**主问题（Master problem）**处理一部分决策变量，而**子问题（Subproblem）**处理另一部分决策变量。主问题和子问题通过解和**Benders割（Benders cuts）**交换信息。

- 「**主问题**」：原始问题的松弛（包含原始问题的部分约束和变量），通常是一个混合整数规划问题。
- 「**子问题**」：子问题是一个线性规划问题或其他形式的优化问题，子问题的输入为主问题的求解结果。
- 「**Benders割**」：通过求解子问题，可以得到Benders割,即线性不等式。它们被添加到主问题中，以改善主问题解的质量。
- 「**迭代&收敛**」：在每一次迭代中，先求解主问题（为子问题提供输入），然后求解子问题（为主问题提供 Benders 割）。当主问题解的质量无法进一步提升时算法收敛。



经典 Benders 分解^[2]

- 子问题为线性规划

组合 Benders 分解^[3]

- 子问题为线性方程组
- 组合 Benders 割

基于逻辑的 Benders 分解^[4]

- 子问题为任意形式，如约束规划（Constraint Programming）

算法设计

问题分解

■ ...

■ 因此可只考虑分配关系：

□ 工序 $\leftrightarrow$ 工人（目标函数）

工序 $\leftrightarrow$ 工位（可行性）

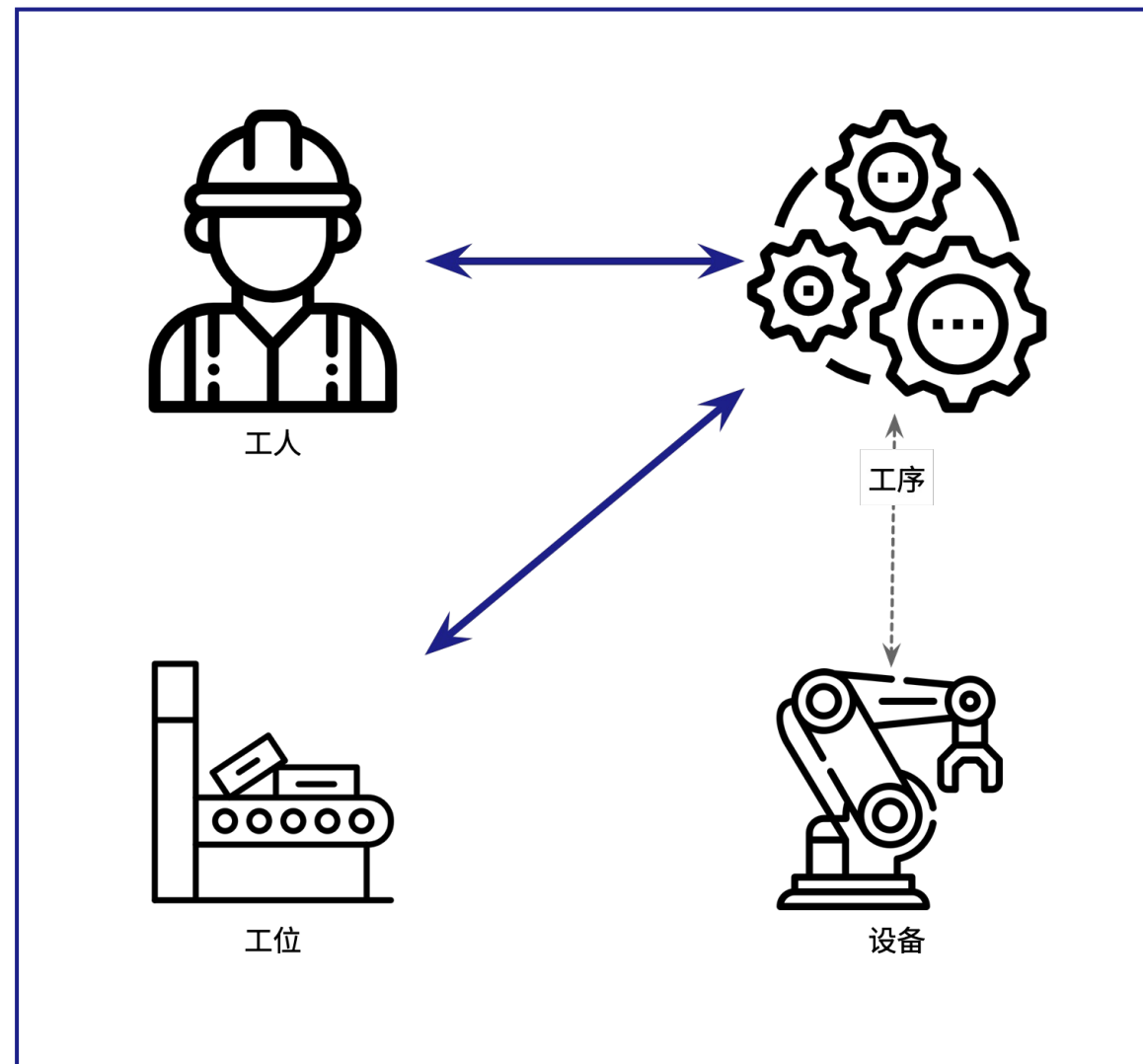
💡 有如下两种分解策略，以解决 ①「**数学模型规模巨大**」：

自顶向下分解

- 主问题：工序 $\leftrightarrow$ 工人
- 子问题：工序 $\leftrightarrow$ 工位（排序）
- 适用于工序间依赖关系简单（拼接工序少）的算例

自底向上分解

- 主问题：工序 $\leftrightarrow$ 工位（排序）
- 子问题：工序 $\leftrightarrow$ 工人
- 适用于工序间依赖关系复杂（拼接工序多）的算例



算法设计

主问题

建模&求解:



集合

- ☐ W
- ☐ P

工人
工序

变量

- ☐ $x_{wp} \in \{0,1\}, \forall w \in W, p \in P$
- ☐ $y_w \in \mathbb{R}^+, \forall w \in W$
- ☐ $n_p \in \{0,1\}, \forall p \in P$

工人 $\leftrightarrow$ 工序
工人的节拍
工序是否被拆分

参数

- ☐ $standTime_p$
- ☐ $effi_{wp}$
- ☐ $skillCap_{wp}$
- ☐ $catCap_{wp}$
- ☐ W_p
- ☐ $\bar{w}_p$
- ☐ $volRate$
- ☐ $maxWpO$
- ☐ $maxSpO$
- ☐ $maxSplit$

工序标准工时
工人效率
是否具备所需技能
是否具备技能类型
具备所需技能的工人集
预先指定的分配关系
节拍波动率
工序最大工人数量
工序最大工位数量
最大拆分工序数量

Minimize

- ☐ $\max_{w \in W} y_w$
- ☐ $\sum_{w \in W} y_w$
- ☐ $\sum_{w_1, w_2 \in W} |y_{w_1} - y_{w_2}|$
- ✓ 0

最小化最大节拍
最小化总（平均）节拍
基尼系数^[8]
可行性问题

Subject to (不拆分工序)

- ☐ $y_w = \frac{standTime_p}{effi_{wp}} \cdot x_{wp}, \forall w \in W$ * 工人节拍
- ☐ $\sum_{w \in W} x_{wp} = 1, \forall p \in P$ * 每个工序一个工人
- ☐ $\sum_{w \in W_p} x_{wp} = 1; \sum_{w \in W - W_p} x_{wp} = 0, \forall w \in W, W_p \neq \emptyset$ * 优先分配同技能
- ☐ $\sum_{p \in P} x_{wp} \geq 1, \forall w \in W$ 每个工人都有工序
- ☐ $\sum_{w \in W} x_{wp} \leq \max(skillCap_{wp}, catCap_{wp}), \forall p \in P$ 技能要求
- ☐ $x_{w_p p} = 1; x_{wp} = 0, \forall p \in P, w \in W - \{w_p\}$ 预先指定分配
- ☐ $\frac{\sum_{w \in W} y_w (1 - volRate)}{|W|} \leq y_w \leq \frac{\sum_{w \in W} y_w (1 + volRate)}{|W|}, \forall w \in W$ 波动率
- ☐ $x_{wp} \in \{0,1\}, y_w \in \mathbb{R}^+, \forall w \in W, p \in P$ 变量定义域

Subject to (拆分工序)

- ☐ $y_w = \frac{standTime_p}{effi_{wp}} \cdot \frac{x_{wp}}{\sum_{w \in W} x_{wp}}, \forall w \in W$ * 工人节拍
- ☐ $1 \leq \sum_{w \in W} x_{wp} \leq \max(maxWpO, maxSpO), \forall p \in P$ * 工序工人数量限制
- ☐ $\sum_{w \in W_p} x_{wp} \geq 1; \sum_{w \in W - W_p} x_{wp} = 0, \forall w \in W, W_p \neq \emptyset$ * 优先分配同技能
- ☐ $\sum_{w \in W} x_{wp} \geq 2 \Rightarrow n_p = 1, \sum_{p \in P} n_p \leq maxSplit, \forall p \in P$ 被拆分的工序数量
- ☐ $n_p \in \{0,1\}, \forall p \in P$ 变量定义域

算法设计

子问题

建模&求解:



Google OR-Tools

集合

- ☐ S' 工位
- ☐ $\bar{S}'$ 有固定设备工位
- ☐ M 设备
- ☐ M' $isMachineNeed = True$
- ☐ $monoM$ 独占工位设备
- ☐ M_s 工位固定设备
- ☐ $S = S' \cdot maxCycle$ 拓展工位

💡 通过将 S' 拓展为 $S = S' \cdot maxCycle$, 环形生产线平衡问题转化为直线生产线平衡问题, ②「**环形生产线建模**」得以解决。

变量

- ☐ $x_{ps} \in \{0,1\}, \forall p \in P, s \in S$ 工序 $\leftrightarrow$ 工位 (拓展)
- ☐ $x'_{ps} \in \{0,1\}, \forall p \in P, s \in S'$ 工序 $\leftrightarrow$ 工位
- ☐ $x_{ws} \in \{0,1\}, \forall w \in W, s \in S'$ 工人 $\leftrightarrow$ 工位
- ☐ $x_{ms} \in \{0,1\}, \forall m \in M, s \in S'$ 设备 $\leftrightarrow$ 工位
- ☐ $z_{cs} \in \{0,1\}, \forall c \in C, s \in S'$ 工位在某圈被使用
- ☐ $z_s \in \mathbb{Z}_0^+, \forall s \in S'$ 工位重复入站次数

参数

- ☐ $\bar{x}_{wp}$ 主问题输入
- ☐ $immePre$ 工序顺序
- ☐ $maxSpW$ 工人最大工位数量
- ☐ $maxMpS$ 工位最大设备数量
- ☐ $maxRS$ 最大重复入站次数

💡 拆分工序情况下, 对主问题的解进行处理, 以解决 ③「**工序拆分建模**」:

例: 原有工序集合为 $\{1, 2, 3, 4, 5, 6\}$, 其中 $\{4, 5\}$ 被分别拆分给两个工人。那么, 拓展的工序集合为 $\{1, 2, 3, 4, 5, 6, | 7, 8\}$, 其中 7 具有和 4 完全相同的特征, 8 具有和 5 完全相同的特征。即:

- ☐ 由主问题确定的执行该工序的工人
- ☐ 原始算例数据中工序的所有参数 (如: 工序的紧前工序、工序所需设备等)

完成以上操作后, 拆分工序和不拆分工序的情况可使用**统一的子问题**求解。并且, 设计的算法增强策略 (快速子问题不可行性证明和局部分支) 可以兼容以上拓展模型。

Subject to

变量链接

- ☐ $x_{ps'} \Rightarrow x'_{ps}, \forall p \in P, s' \in S_s, s \in S$ 原始和拓展工位序列
- ☐ $x_{ws} = \bar{x}_{wp} \cdot x'_{ps}, \forall w \in W, s \in S', p \in P$ 工人 $\leftrightarrow$ 工位 $\leftrightarrow$ 工序

工序约束

- ☐ $\sum_{s \in S} x_{ps} = 1, \forall p \in P$ 每个工序一个工位

算法设计

子问题

建模&求解:



Google OR-Tools

集合

- $\square S'$ 工位
- $\square \bar{S}'$ 有固定设备工位
- $\square M$ 设备
- $\square M'$ $isMachineNeed = True$
- $\square monoM$ 独占工位设备
- $\square M_s$ 工位固定设备
- $\square S = S' \cdot maxCycle$ 拓展工位

💡 通过将 S' 拓展为 $S = S' \cdot maxCycle$, 环形生产线平衡问题转化为直线生产线平衡问题, ②「**环形生产线建模**」得以解决。

变量

- $\square x_{ps} \in \{0,1\}, \forall p \in P, s \in S$ 工序 $\leftrightarrow$ 工位 (拓展)
- $\square x'_{ps} \in \{0,1\}, \forall p \in P, s \in S'$ 工序 $\leftrightarrow$ 工位
- $\square x_{ws} \in \{0,1\}, \forall w \in W, s \in S'$ 工人 $\leftrightarrow$ 工位
- $\square x_{ms} \in \{0,1\}, \forall m \in M, s \in S'$ 设备 $\leftrightarrow$ 工位
- $\square z_{cs} \in \{0,1\}, \forall c \in C, s \in S'$ 工位在某圈被使用
- $\square z_s \in \mathbb{Z}_0^+, \forall s \in S'$ 工位重复入站次数

参数

- $\square \bar{x}_{wp}$ 主问题输入
- $\square immePre$ 工序顺序
- $\square maxSpW$ 工人最大工位数量
- $\square maxMpS$ 工位最大设备数量
- $\square maxRS$ 最大重复入站次数

Subject to (续)

工序约束 (续)

- $\square \sum_{s \in S} s \cdot x_{p_1 s} \leq \sum_{s \in S} s \cdot x_{p_2 s}, \forall (p_1, p_2) \in immePre$ 工序先后关系
- $\square \sum_{s' \in S_{sp}} x_{ps'} = 1, p \in P$ 指定工序工位

工位约束

- $\square \sum_{w \in W} x_{ws} \leq 1, \forall s \in S'$ 每个工位至多一个工人

工人约束

- $\square 1 \leq \sum_{s \in S'} x_{ws} \leq maxSpW, \forall w \in W$ 工人工位数量约束

设备约束

- $\square x_{ps} \leq x_{m, (s-1) \% |S'| + 1}, \forall p \in P, s \in S$ 工位要有所需设备
- $\square \sum_{m \in M'} x_{ms} \leq maxMpS, \forall s \in S$ 工位设备数量上限
- $\square x_{m's} \leq 1 - x_{ms}, \forall s \in S', m \in monoM, m' \in M - \{m\}$ 设备独占工位
- $\square x_{ms} = 1, \forall s \in S', M_s \neq \emptyset, m \in M_s$ 固定设备约束
- $\square x_{ms} = 0, \forall s \in S', M_s \neq \emptyset, m \in M - M_s$ 固定设备约束

算法设计

子问题

建模&求解:



Google OR-Tools

集合

- S' 工位
- $\bar{S}'$ 有固定设备工位
- M 设备
- M' $isMachineNeed = True$
- $monoM$ 独占工位设备
- M_s 工位固定设备
- $S = S' \cdot maxCycle$ 拓展工位

💡 通过将 S' 拓展为 $S = S' \cdot maxCycle$, 环形生产线平衡问题转化为直线生产线平衡问题, ②「环形生产线建模」得以解决。

变量

- $x_{ps} \in \{0,1\}, \forall p \in P, s \in S$ 工序 $\leftrightarrow$ 工位 (拓展)
- $x'_{ps} \in \{0,1\}, \forall p \in P, s \in S'$ 工序 $\leftrightarrow$ 工位
- $x_{ws} \in \{0,1\}, \forall w \in W, s \in S'$ 工人 $\leftrightarrow$ 工位
- $x_{ms} \in \{0,1\}, \forall m \in M, s \in S'$ 设备 $\leftrightarrow$ 工位
- $z_{cs} \in \{0,1\}, \forall c \in C, s \in S'$ 工位在某圈被使用
- $z_s \in \mathbb{Z}_0^+, \forall s \in S'$ 工位重复入站次数

参数

- $\bar{x}_{wp}$ 主问题输入
- $immePre$ 工序顺序
- $maxSpW$ 工人最大工位数量
- $maxMpS$ 工位最大设备数量
- $maxRS$ 最大重复入站次数

Subject to (续)

解决 ④「重复入站建模」

重复入站约束

- $x_{p,s+c|S'|} \Rightarrow z_{cs}, \forall s \in S', c \in C, p \in P$ 是否入站
- $z_s = \max\{0, \sum_{c \in C} z_{cs} - 1\}, \forall s \in S'$ 重复入站次数
- $z_s \leq 2, \forall s \in S' - \bar{S}'$ 无固定设备重复入站
- $\sum_{s \in S'} z_s \leq maxRS$ 总重复入站

变量定义域

- $x_{ps} \in \{0,1\}, \forall p \in P, s \in S$ 变量定义域
- $x'_{ps}, x_{ws}, x_{ms} \in \{0,1\}, \forall p \in P, s \in S', w \in W, m \in M$ 变量定义域
- $z_{cs} \in \{0,1\}, \forall c \in C, s \in S'$ 变量定义域
- $z_s \in \mathbb{Z}_0^+, \forall s \in S'$ 变量定义域

子问题为可行性问题 (无目标函数)。

- 子问题可行 $\Rightarrow$ 主问题已达到全局最优;
- 子问题不可行 $\Rightarrow$ 生成「组合 Benders 割」并将其传递给主问题。

算法设计

组合 Benders 割

主问题 $\rightarrow$ 子问题（右图 **Information**）

□ 工人 $\leftrightarrow$ 工序, $\bar{x}_{wp}, \forall w \in W, p \in P$

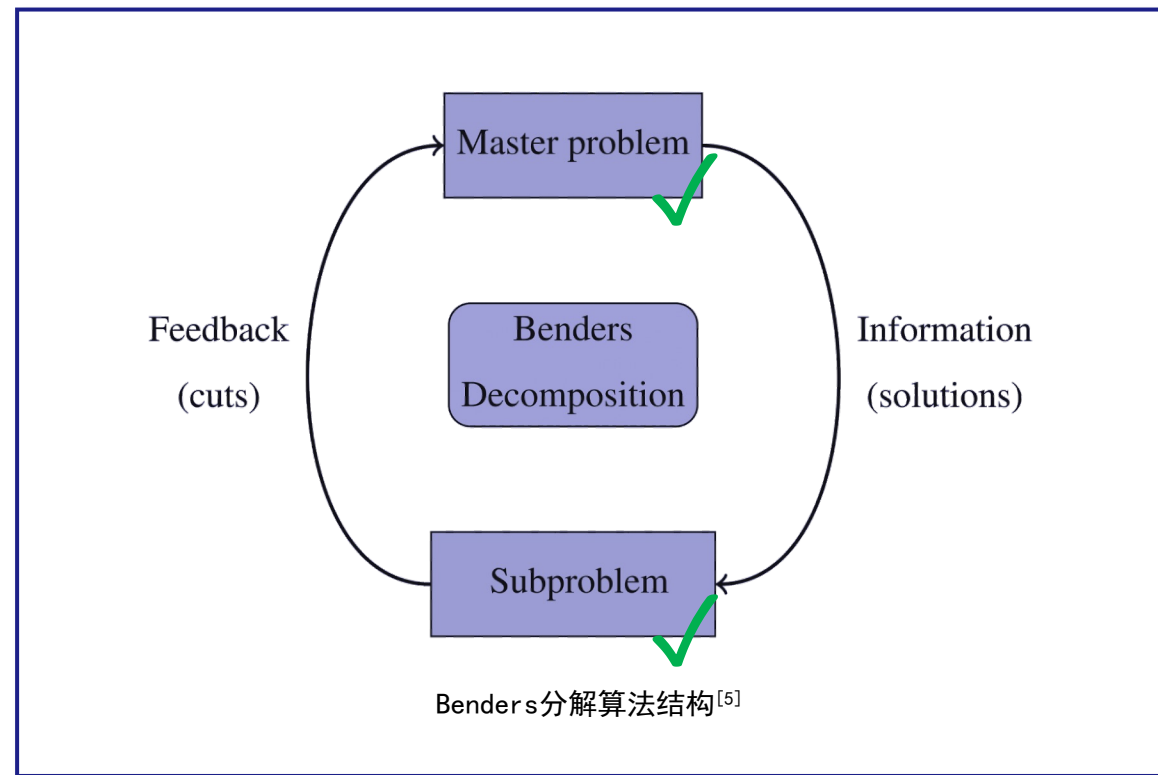
子问题 $\rightarrow$ 主问题（右图 **Feedback**）

□ 组合 Benders 割, 「汉明距离」与 $\bar{x}_{wp}$ 为 1:

$$\sum_{\{(w,p)|\bar{x}_{wp}=0\}} x_{wp} + \sum_{\{(w,p)|\bar{x}_{wp}=1\}} (1 - x_{wp}) \geq 1$$

但是, 仍有以下问题:

- 尽管 CP 求解子问题相比求解完整的 MIP 和 MIP 形式的子问题较快, 但仍需要很长时间证明子问题不可行;
- 以上形式的组合 Benders 割较弱 (no good cut), 仅能从主问题解空间中移除一个解。



经典 Benders 分解^[2]

- 子问题为线性规划

组合 Benders 分解^[3]

- 子问题为线性方程组
- 💡 组合 Benders 割

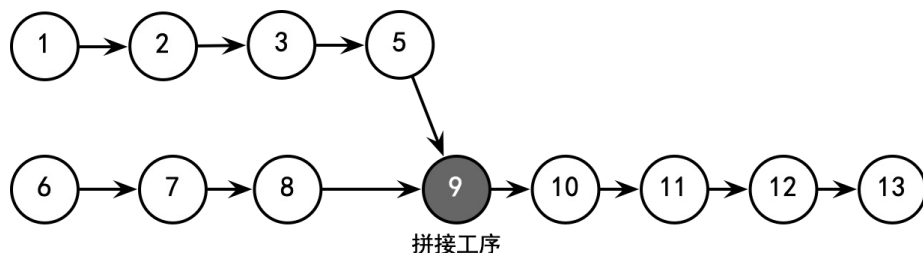
基于逻辑的 Benders 分解^[4]

- 💡 子问题为任意形式, 如约束规划 (Constraint Programming)

算法设计

算法增强：（1）快速不可行性证明

「**拓扑排序**」（**Topological sorting**）是将有向图中所有顶点排成一个线性序列，使得图中任意一对顶点 u 和 v ，若 $\langle u, v \rangle$ 是有向图的边，则在线性序列中 u 出现在 v 之前。



根据拓扑排序的定义，上图拓扑排序有：

拓扑排序1 [1, 6, 2, 7, 3, 8, 5, 9, 10, 11, 12, 13]

拓扑排序2 [6, 1, 7, 2, 8, 3, 5, 9, 10, 11, 12, 13] ...（不唯一）

集合形式 [{1, 6}, {2, 7}, {3, 8}, {5}, {9}, {10}, {11}, {12}, {13}]

- 1和6任意顺序执行，2和7任意顺序执行，...，13最后执行
- 拓扑排序是一个「可行」的工序执行顺序（不考虑其他约束）
- 💡 若已知工序分配的工人，可根据拓扑排序计算所需工位数量

使用 $\bar{x}_{wp}$ 和拓扑排序计算「所需工位数量」

工序顺序 [{1, 6}, {2, 7}, {3, 8}, {5}, {9}, {10}, {11}, {12}, {13}]

工序工人 [{A, B}, {A, B}, {A, A}, {A}, {C}, {D}, {D}, {E}, {F}]

{1, 6} ({A, B}) 需要工位数量: 2 (同一工位不能有多个人)

{2, 7} ({A, B}) 需要工位数量: 1 (2和1, 或6和7可共享工位)

{3, 8} ({A, A}) 需要工位数量: 0 (3和8可与2共享工位)

...

重复以上步骤直至「所需工位总数 > 工位数量 + 最大重复入站次数」，并记录此时的工序集合 p_{min} 。此时，主问题的解 $\bar{x}_{wp}$ 必然导致子问题不可行。

- 快速证明子问题不可行（无需求解 CP 模型）

⇒ 更多 的组合 Benders 割

- 导致子问题不可行的 $(p_{min}, |p_{min}| \leq |P|)$

⇒ 更强 的组合 Benders 割



算法设计

算法增强：（2）增强组合 Benders 割

- 原始的组合 Benders 割可以通过快速不可行性证明得到的 p_min 进行增强。
- 原始的组合 Benders 割：

$$\sum_{\{(w,p)|\bar{x}_{wp}=0,w\in W,p\in P\}} x_{wp} + \sum_{\{(w,p)|\bar{x}_{wp}=1,w\in W,p\in P\}} (1 - x_{wp}) \geq 1$$

- 增强的组合 Benders 割：

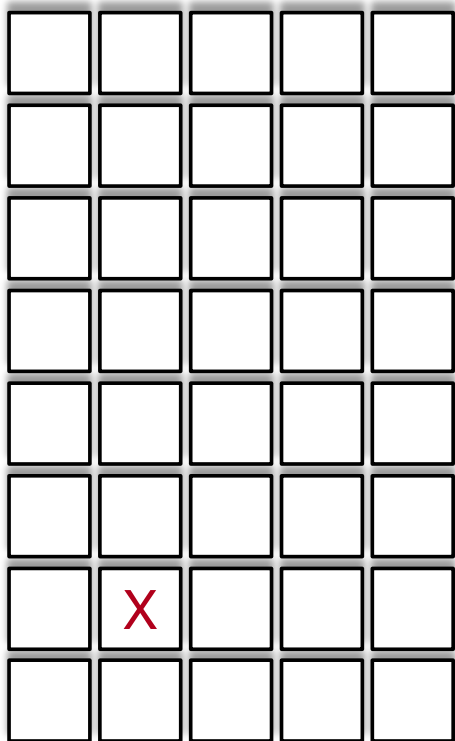
$$\sum_{\{(w,p)|\bar{x}_{wp}=0,w\in W,p\in p_min\}} x_{wp} + \sum_{\{(w,p)|\bar{x}_{wp}=1,w\in W,p\in p_min\}} (1 - x_{wp}) \geq 1$$

- $p \in P \rightarrow p \in p_min, |p_min| \leq |P|$ ，增强的组合 Benders 割可以更精准地定位导致子问题不可行的原因。

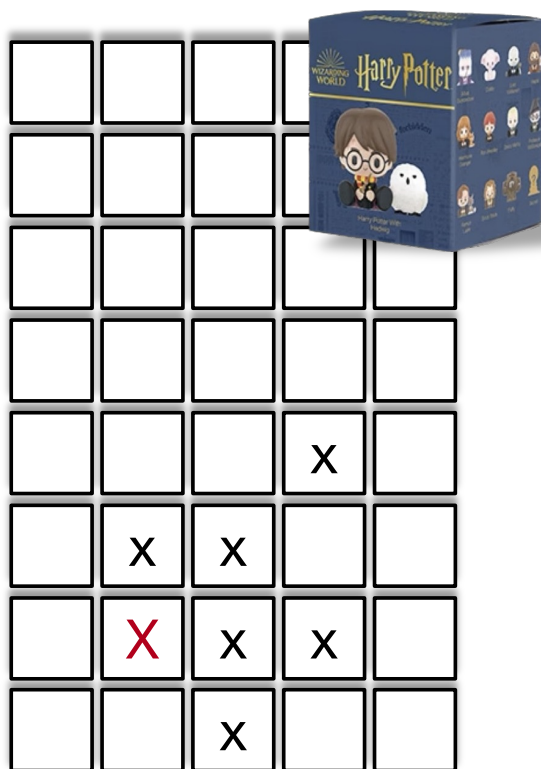
算法设计

算法增强： (3) 局部分支 (Local Branching) [6][7]

Exploitation VS Exploration



(1) 组合 Benders 割



(2) 增强的组合 Benders 割



(3) 局部分支

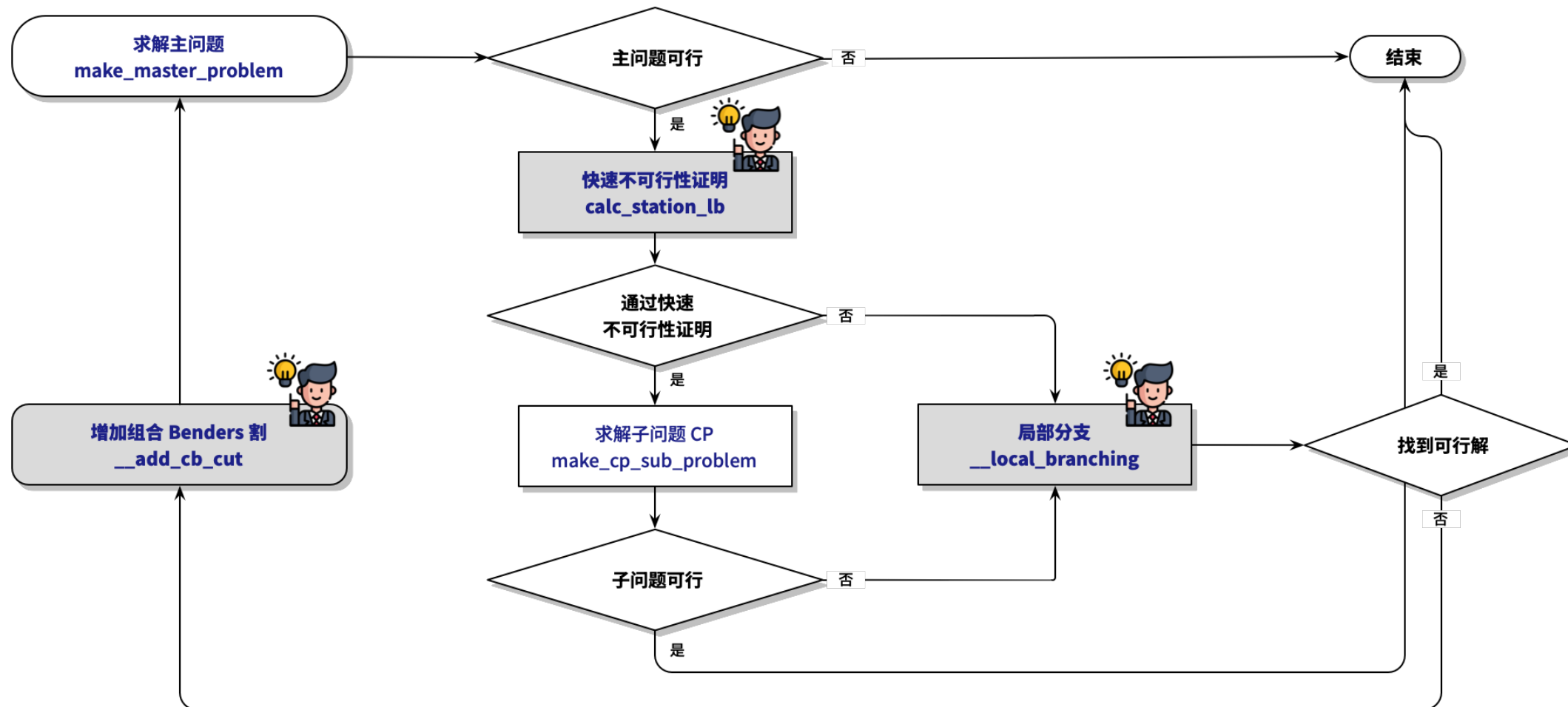
「局部分支」通过增加以下不等式实现：

$$\sum_{\{(w,p)|x_{wp}=0\}} x_{wp} + \sum_{\{(w,p)|x_{wp}=1\}} (1 - x_{wp}) \geq k$$

由于以上不等式为 invalid cut (可能移除可行解)，局部分支操作结束后从模型中删除所有以上割平面。

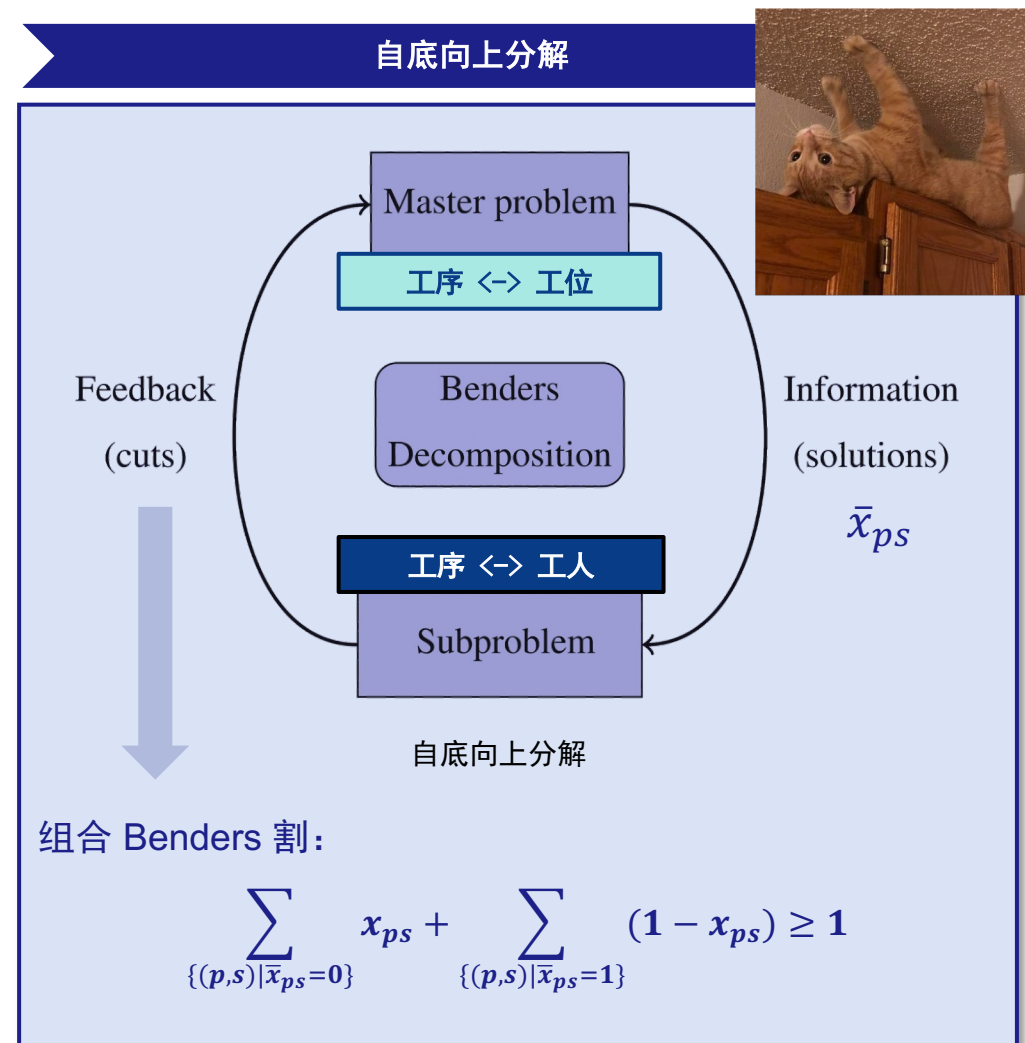
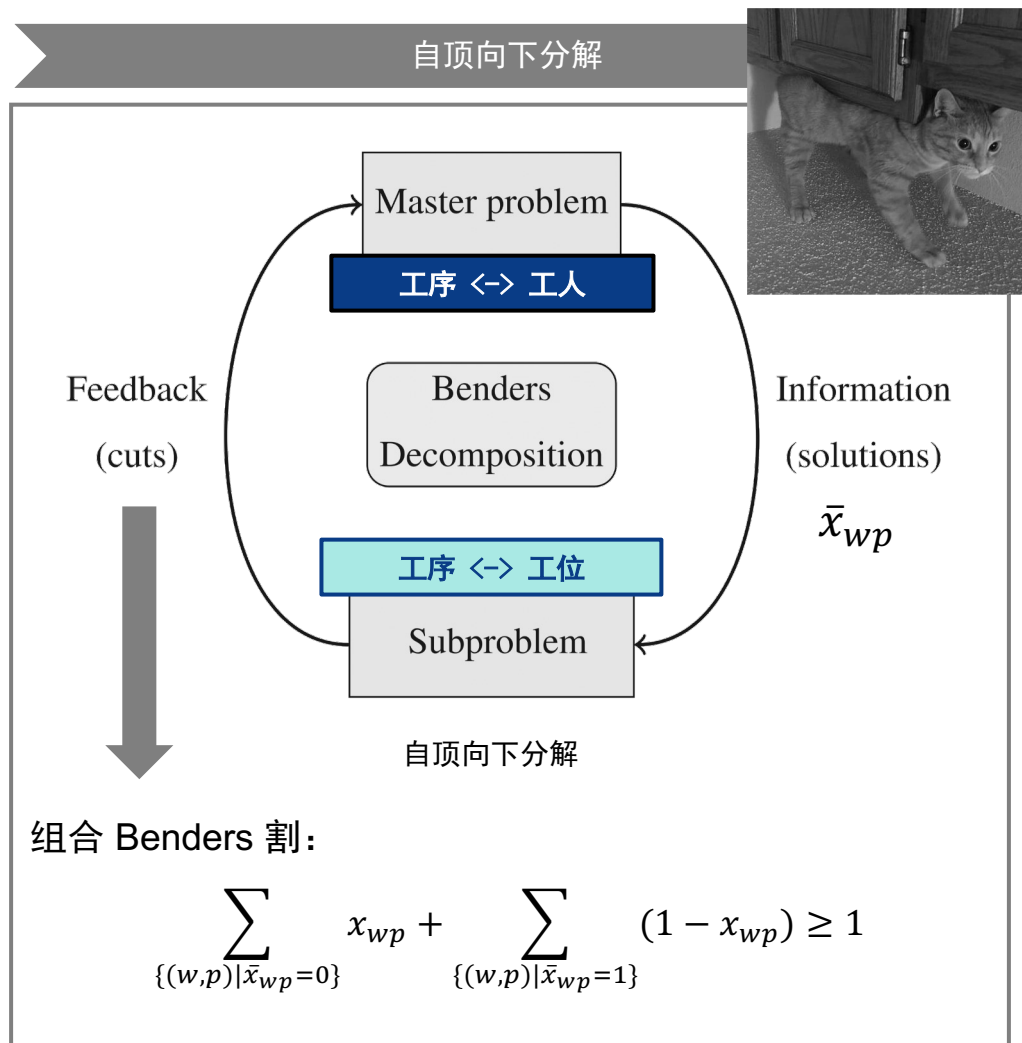
算法设计

算法流程



算法设计

自底向上方法

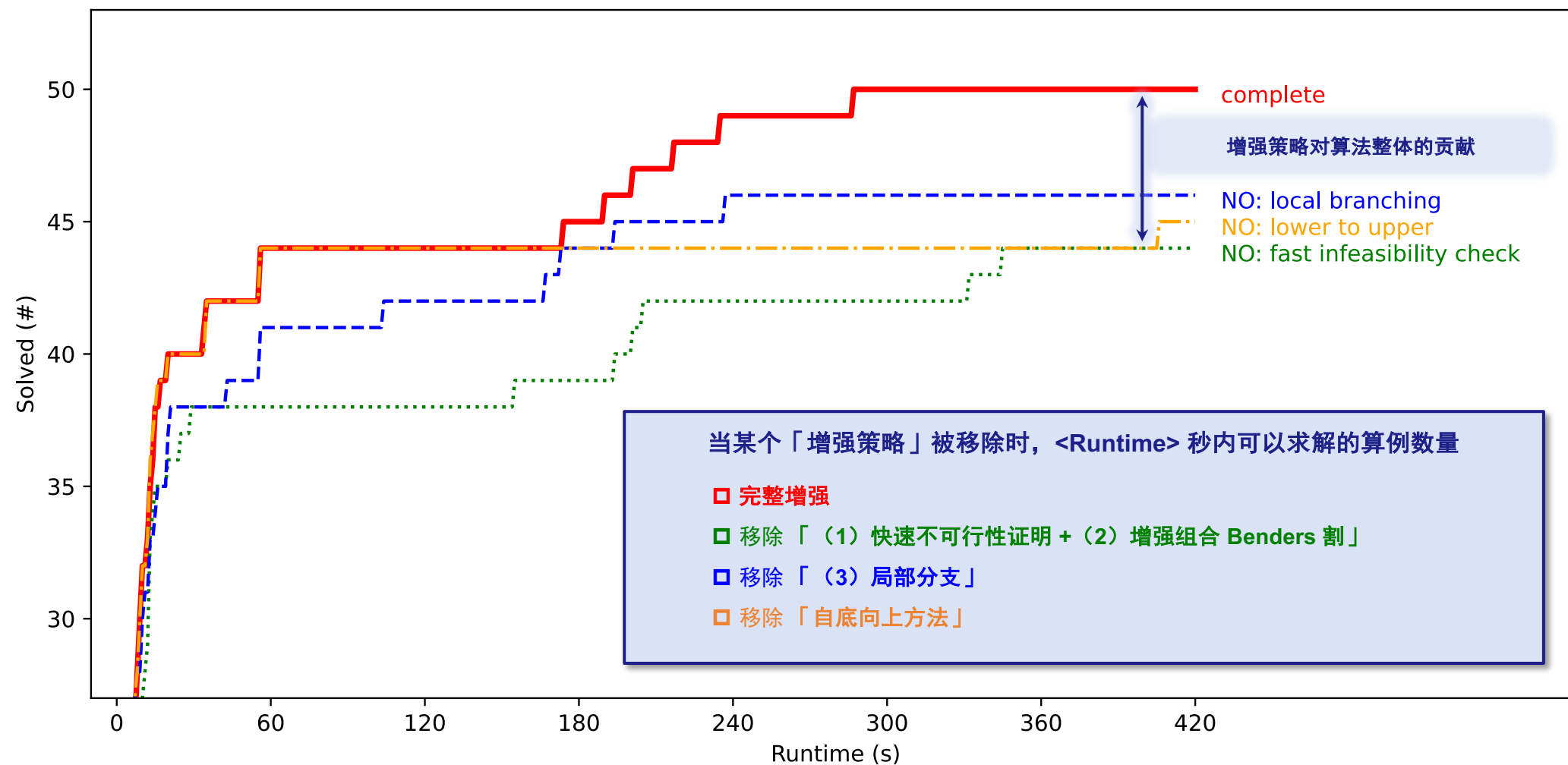


04 / 数值结果

“华为云杯”2023人工智能应用创新大赛

数值结果

增强策略效果



数值结果

整体表现

非公开算例成绩：3.0018（排名：2），公开算例表现（4C4G）： $2.6800 = (0.2911*29+3.9853*10+0.2507*10+10*11)/60$

	算例数量	解数量 (0 / 最大节拍)	平均求解时间 (Min.0)	真实评价指标 (Min.0)	平均求解时间 (Min.最大节拍)	真实评价指标 (Min.最大节拍)
不拆分 / 简单	29	29 / 29	5.72 秒	0.3668	7.83 秒	0.2911
不拆分 / 困难	16	10 / 8	195.75 秒	3.9853	224.62 秒	5.1879
拆分 / 简单	10	10 / 10	8.87 秒	0.3703	6.58 秒	0.2507
拆分 / 困难	2	0 / 0	-	-	-	-

05 / 结论与展望

“华为云杯”2023人工智能应用创新大赛

结论与展望

算法优势

	基于 Benders 分解的 精确算法	基于规则的 启发式算法	引入随机的 元启发式算法
解的最优性	可以保证全局最优	无法保证	无法保证
解的可行性	可以保证	取决于具体算法设计	取决于具体算法设计
速度	2/3 以上公开算例秒级给出最优解或可行解	快	取决于具体算法设计
拓展性	易增加或减少约束	取决于算法设计和约束类型	取决于算法设计和约束类型

结论与展望

■ 结论

- [模型算法] 针对「**环形生产线平衡**」问题，基于「**CBD 和 LBBD**」开发了精确求解算法。
- [增强策略] 实现了「**快速不可行性证明、增强组合 Benders 割、和局部分支**」三个算法增强。
- [分解策略] 实现了「**自顶向下**」分解（简单拓扑）和「**自底向上**」分解（复杂拓扑）。
- [数值结果] 求解了 5/6 的公开算例（< 420 秒），2/3 的公开算例所需求解时间「**< 10 秒**」。

■ 展望

- [算法框架] 使用 Branch-and-check 在单个分支定界树上进行搜索，以加速主问题。
- [增强策略] 将快速不可行性证明、增强组合 Benders 割、和局部分支用于自底向上的分解策略。
- [最优性割] 目前自底向上的分解策略仅实现了可行性割，保证全局最优还需要实现最优性割。

参考文献

1. Boysen, N., Schulze, P., & Scholl, A. (2022). **Assembly line balancing: What happened in the last fifteen years?** *European Journal of Operational Research*, 301(3), 797–814. <https://doi.org/10.1016/j.ejor.2021.11.043>
2. Benders, J. F. (1962). **Partitioning procedures for solving mixed-variables programming problems.** *Numerische Mathematik*, 4(1), 238–252. <https://doi.org/10.1007/BF01386316>
3. Codato, G., & Fischetti, M. (2006). **Combinatorial Benders' cuts for mixed-integer linear programming.** *Operations Research*, 54(4), 756–766. <https://doi.org/10.1287/opre.1060.0286>
4. Hooker, J. N., & Ottosson, G. (2003). **Logic-based Benders decomposition.** *Mathematical Programming*, 96(1), 33–60. <https://doi.org/10.1007/s10107-003-0375-9>
5. Rahmaniani, R., Crainic, T. G., Gendreau, M., & Rei, W. (2017). **The Benders decomposition algorithm: A literature review.** *European Journal of Operational Research*, 259(3), 801–817. <https://doi.org/10.1016/j.ejor.2016.12.005>
6. Fischetti, M., & Lodi, A. (2003). **Local branching.** *Mathematical Programming*, 98(1), 23–47. <https://doi.org/10.1007/s10107-003-0395-5>
7. Rei, W., Cordeau, J.-F., Gendreau, M., & Soriano, P. (2009). **Accelerating Benders Decomposition by Local Branching.** *INFORMS Journal on Computing*, 21(2), 333–345. <https://doi.org/10.1287/ijoc.1080.0296>
8. Tsang, M. Y., & Shehadeh, K. S. (2022). **Convex Fairness Measures: Theory and Optimization (arXiv:2211.13427).** *arXiv*. <http://arxiv.org/abs/2211.13427>



On the **shoulders of giants**

Thanks

基于 Benders 分解的环形生产线平衡：
自顶向下和自底向上方法

郭鹏辉（Min），<https://guo.ph>
苏州工业园区人工智能产业园，2023-9-26

